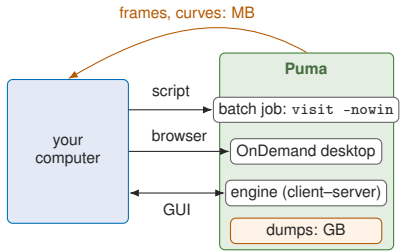
VISIT ON THE CLUSTER

Where the data are: VisIt scripts in batch jobs on Puma, a desktop on a compute node, X11, and client–server; a parallel engine; resampling, averages and movies without downloading the run

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part V • Chapter 31



measured on IB/11 and IBFE/10 with VisIt 3.5.0 (one machine, no cluster); Puma's side from UArizona's documentation and the 2026 notes, to be run there

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

- Say which of four ways to run VisIt with a cluster fits a task: a script in a batch job, a desktop on a compute node, X11 forwarding, or client–server.
- Get VisIt on Puma (the module, or your own install) and run a VisIt script there without a display.
- Turn a script tested on your computer into a batch job that writes frames, averages or resampled data next to the run, and check what it wrote.
- Start a parallel compute engine, and know when it helps.
- Use the 2020 cluster tutorial's recipes (IBFE meshes, resampling, VTK databases, averages over time) in VisIt 3.5.

> Move the analysis to the data, not the data to the analysis: a 3D run's dumps are gigabytes, its frames and curves are megabytes.

WHERE THIS CHAPTER COMES FROM

- G. Hobson, “Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster” (Scripps Institution of Oceanography, UC San Diego, 23 September 2020; “A contribution to the IBAMR Tutorials Research Group”, 32 slides): VisIt 3.1.2 installed from the tarball on UNC’s Dogwood and Longleaf clusters, scripts recorded in the GUI and run with `visit -cli -nowin -s`, the rubber band, IBFE meshes, 3D resampling, VTK databases, averages over time, restarting a job.
- “Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster” (UArizona, “Based on a tutorial created by Gabrielle Hobson”, 8 slides; undated, its file made in October 2023): the same for UArizona, the tarball uploaded with FileZilla (“it will be too large to do through the open on demand virtual desktop”), and “MAKE SURE YOU ARE IN INTERACTIVE MODE!!!!”.
- “3B – Analyzing your IBFE simulation” (2018), slides 3–9, and “8 – Visit from command line and Python” (2018): the Killdevil cluster’s `visit/2.12.3` module, `visit -cli -nowin -s`, building VisIt with `build_visit --mesa`, and “The visit installed on killdevil cannot read .ex2 files”.
- UArizona HPC documentation (read 17 October 2026): the VisIt page (an Open OnDemand desktop; client–server through the HPC VPN) and the visualization page (“it is also available as a module on our system”; X11 with GLX); the 2026 notes from runs on Puma (`visit/3.4.2`; Puma the only cluster).

New here: every recipe in VisIt 3.5’s Python 3, a batch script for frames (`examples/analysis/visit_frames.slurm`), the parallel engine and the memory of a long render measured, the 2020 workarounds re-tested, and what UArizona’s page says that no longer holds. What needs Puma itself is marked so: this chapter’s measurements were made on one desktop PC, not on Puma.

WHAT THE OLD MATERIAL ASSUMED, AND WHAT CHANGED

then	now
UNC's Killdevil, Dogwood, Longleaf; UArizona's Ocelote and El Gato	Puma (Rocky Linux 9 since early 2025); Ocelote and El Gato retired in 2026; a new cluster, Lynx, announced with the same system
download the VisIt tarball, upload it ("filezilla or something"), untar	a `visit` module on Puma (3.4.2 in 2026), or your own 3.5.0 install (Rocky 9 build) (slide 7)
scripts in VisIt's `bin` folder, run from there	anywhere: give `visit` its full path, the script its data paths
"MAKE SURE YOU ARE IN INTERACTIVE MODE!!!!"	interactive for a test, a batch job for the work (slides 10–11)
`tar -xvf`, `-cli -nowin -s` (en dashes from the slide software)	`tar -xvf`, `visit -cli -nowin -s`: type them, do not paste them from slides
Python 2 (VisIt 2.x to 3.1, so the 2020 tutorial's 3.1.2 too) and the recorder's Python	VisIt 3.2 and later embed Python 3 (3.13 in 3.5.0): `print()` with brackets, `range`, integer division `//`; `-py2to3` converts old scripts on the fly, within limits
a loop that advances before its first save	start at dump 0 (chapter 30, slide on your own loop)
"VisIt has trouble reading .VTK databases" on the cluster	VisIt 3.5 opens a series of `.vtk` files directly (slide 18)
"cannot read .ex2 files" (Killdevil, VisIt 2.12.3)	VisIt 3.x reads ExodusII (chapter 28)

The decks of the previous slide; Puma: UArizona HPC documentation (updates page) and the 2026 notes; Python 3 from VisIt 3.2.0 on: its release notes.

WHY VISIT ON THE CLUSTER

The 2020 tutorial's list (UArizona's copy adds the last one):

- "Speed! The post-processing time is much faster"
- "Your simulation data is already on the cluster, so no time spent downloading it"
- "Fewer human error issues when everything is automated"
- "Scripts make it easier to document and share your analysis"
- "Easy to make consistent images for publication"

The sizes, from this series' runs:

run	dumps	what you take home
IB/11 heart-tube pump (2D, 51 dumps)	129 MB (133 with the structure)	51 frames: 4.1 MB; the movie 0.56 MB
IBFE/10 flow tank (3D, 21 dumps)	95 MB	curves and pictures: kB to MB
a production 3D run (more points, more dumps, finer grid; an estimate)	tens of GB	the same: kB to MB

G. Hobson (2020), slide 3; UArizona's version, slide 2. Sizes: `du -sh` of the runs' `viz_IB2d` and `viz_IB3d` folders and of the 51 PNG frames; the movie: chapter 30.

FOUR WAYS TO USE VISIT WITH A CLUSTER

way	what runs where	what you need	use it for
1. a script in a batch job	everything on a compute node, no window (`visit -nowin -cli -s`)	a script that works, tested on a few dumps	movies, averages, resampling of whole runs; the default (slides 8–11)
2. an Open OnDemand desktop	the GUI on a compute node, seen in your browser	a browser and an allocation	exploring a run interactively; the movie wizard (slide 20)
3. X11 forwarding	the GUI on a compute node, drawn on your screen over `ssh -X`	an X server (XQuartz on a Mac), GLX	quick looks; slower than 2 (slide 21)
4. client–server	the GUI on your computer, the engine on a compute node	the HPC VPN, the same VisIt version on both ends, a host profile	interactive work on big data with a fast local GUI (slides 22–23)

UArizona's advice for graphical programs: "We suggest you first try the Desktop application of OnDemand", and X11 only as the alternative, "in most cases the performance is not as good".

UArizona HPC documentation, Visualization and VisIt pages (read 17 October 2026).

GETTING VISIT ON PUMA

- The module. UArizona: VisIt “is also available as a module on our system”; the 2026 runs found `visit/3.4.2`, with a Python 3 CLI. Modules load only on compute nodes (chapter 1), so look from an `interactive` session:

```
interactive -a <your group>          # a compute node (chapter 1)
module avail visit                   # which versions
module load visit
visit -version
```

- Your own install, for a version the module does not have (this series used 3.5.0): the Linux tarball of the same operating system as the cluster, which for Puma is Rocky 9 (VisIt’s release page: “RockyLinux versions should run on RHEL of the same version”), installed with VisIt’s `visit-install` script (“For linux, the visit-install script is needed to complete an install”) into your home or group folder; run it by its full path, `/groups/<PI>/visit/bin/visit`. (This series simply untarred the Ubuntu 24 tarball, which ran as it was.)
- Match the versions: a script written for 3.5 may need small changes in 3.4 (test it on a few dumps first); for client–server the two ends must agree at least in their first two numbers (slide 23).

UArizona HPC documentation; VisIt’s releases table (3.5.0, April 2026: Ubuntu 24/22, Fedora 40, Debian 12, Rocky 9/8). Not yet run on Puma: `module avail visit` is the first check.

STEP 1: A SCRIPT THAT WORKS ON YOUR COMPUTER

- The 2020 route: set the picture up in the GUI with **Controls** → **Command** recording ("Click Record to begin recording your commands. Once you've completed the steps you want, click Stop."), paste the Python into an editor, keep the lines you need, add a loop that saves every dump (chapter 32 covers the recorder).
- The route of this series: start from a script that already does it, `examples/analysis/visit_movie_frames.py` (frames, chapter 30), `visit_queries.py` (averages, picks, lineouts, chapter 29), `resample_export.py` (a uniform grid, chapter 33), `visit_3d.py` (3D pictures, chapter 27).
- Put every path in one place at the top, as the 2020 tutorial did with its `string_dumps`, `string_mesh`, `string_output`, or pass them as arguments: the same script then runs on your computer and on Puma.
- Test it on the first few dumps (`start=0 end=2`) before the whole run; check the pictures.
- End it with `sys.exit()`: otherwise the CLI stops at its >>> prompt after the script and, in a terminal, waits there. (A batch job, whose input is empty, quits at the prompt by itself: tested with the input from `/dev/null`, which is what Slurm gives a batch script.)

> "These Python scripts are whitespace sensitive." "Errors that pop up in your command line should have the line number in them, this will help with debugging." (2020)

G. Hobson (2020), slides 6–10, 31.

STEP 2: PATHS, AND GETTING THE SCRIPT THERE

What changes between your computer and the cluster is the paths. The UArizona deck (2023) shows a recorded script with Windows paths in it (a student's `C:\Users\...\dumps.visit`); a recorded `OpenDatabase` also carries the host, `localhost:`.

```
# the paths, once, at the top (or from the command line, as the examples/analysis scripts do)
RUN = "/groups/<PI>/<you>/runs/ib11_target_vp"      # on Puma
DUMPS = RUN + "/viz_IB2d/dumps.visit"
LAG = RUN + "/viz_IB2d/lag_data.visit"
OUT = RUN + "/frames"
```

- Copy the script with `scp` or `rsync` (chapter 1), or with the Open OnDemand file browser; it can live anywhere (the UArizona deck puts it in VisIt's `bin` folder and runs `visit` there; the 2020 deck runs VisIt by its full path from the script's folder; nothing needs either).
- Relative paths are relative to the folder you start `visit` in: `visit_frames.slurm` cds into the run's folder first, and `viz_IB2d/dumps.visit` then opens (tested).
- Output folders must exist before VisIt writes into them (chapter 30: VisIt says "Saved" and writes nothing otherwise).

UArizona's deck, slide 7; G. Hobson (2020), slides 9–10.

STEP 3: A TEST ON A COMPUTE NODE

The UArizona deck: “MAKE SURE YOU ARE IN INTERACTIVE MODE!!!!” The login nodes are for editing and submitting, not for running, and Puma’s modules load only on compute nodes.

```
interactive -a <your group>                  # a compute node (interactive -h for more cores or
  time)
module load visit
cd /groups/<PI>/<you>/runs/ib11_target_vp
mkdir -p frames
visit -nowin -cli -s ~/Using-IB-Software/examples/analysis/visit_movie_frames.py -- viz_IB2d frames
   end=2
ls frames                                    # f0000.png f0001.png f0002.png frames.csv
```

- `-nowin`: no window (“the ‘-nowin’ tells it not to open a new window”); `-cli`: the Python command line; `-s`: the script; arguments for the script after `--`.
- Copy the three PNGs home and look at them before the long run.
- The 2020 note: “commands to do with ‘spontaneous states’, like when you click the zoom icon in the GUI, do not work on the cluster”. Set the view with numbers (`View2DAttributes().windowCoords`, `window=` in `visit_movie_frames.py`).

UArizona’s deck, slide 8; G. Hobson (2020), slide 13; the commands as in chapter 1 (not yet run on Puma with VisIt).

STEP 4 (1/2): THE BATCH JOB

```
#!/bin/bash
#SBATCH --job-name=visit-frames
#SBATCH --account=lauram9          # your PI's group
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --mem-per-cpu=5gb          # Puma: 5 GB per core
#SBATCH --time=01:00:00
#SBATCH --output=%x-%j.out
S=$HOME/Using-IB-Software/examples/analysis
VIZ=viz_IB2d                       # a 2D run; 3D: chapter 27
OUT=frames
ARGS="var=Omega lim=auto"
module load visit
# module load ffmpeg               # if `module avail ffmpeg` lists one
cd "$SLURM_SUBMIT_DIR"
mkdir -p "$OUT"; touch "$OUT/.start"
visit -nowin -cli -s "$S/visit_movie_frames.py" -- "$VIZ" "$OUT" $ARGS
n=$(find "$OUT" -maxdepth 1 -name "*.png" -newer "$OUT/.start" | wc -l)
echo "$n new frames in $OUT"
[ "$n" -gt 0 ] || exit 1           # none: the job FAILS
if command -v ffmpeg > /dev/null 2>&1; then
    python3 "$S/make_movie.py" "$OUT" --fps 10 --out "$OUT/movie.mp4"
fi
```

`examples/analysis/visit_frames.slurm`: the frames of a finished run, counted, and a movie where an `ffmpeg` is found (next slide).

`wsl/final/ch31_slurm_local.sh`; the job header as IB/01's `job.slurm` (chapter 1); memory per core: UArizona HPC documentation, "CPUs and Memory".

STEP 4 (2/2): WHAT THE BATCH JOB DOES

- `examples/analysis/visit_frames.slurm`; copy it into the run's folder, set your group, and `sbatch visit_frames.slurm` there; or `sbatch ~/Using-IB-Software/examples/analysis/visit_frames.slurm` from the run's folder (the job works where it was submitted).
- One task: the default VisIt engine is serial. Memory: a 3D slice render held 179–186 MB (slide 16). A core of Puma's standard nodes comes with 5 GB; asking more per core can send the job to one of the few high-memory nodes, with a longer wait.
- It counts the frames this job wrote (newer than `.start`), because VisIt can report success and write nothing (chapter 30); with none the job ends as `FAILED`. It encodes only where an `ffmpeg` exists.
- Run on this machine with a stand-in for `module` (no Slurm here): 51 frames of IB/11 and a 5.1 s movie in 31 s. Not yet run on Puma.

`wsl/final/ch31_slurm_local.sh`; the job header as IB/01's `job.slurm` (chapter 1); memory per core: UArizona HPC documentation, "CPUs and Memory".

STEP 5: CHECK, ENCODE, BRING HOME

```
squeue --me                                  # waiting or running (chapter 1)
cat visit-frames-<jobid>.out                 # "51 new frames in frames", or the error
sacct -j <jobid> --format=JobID,Elapsed,MaxRSS,State
ls frames | head; ls frames/*.png | wc -l
scp -r <you>@filexfer.hpc.arizona.edu:/groups/<PI>/<you>/runs/ib11_target_vp/frames .   # home (
  chapter 1)
```

- Encoding needs ffmpeg. UArizona's VisIt page loads it as a module (`module load gnu8`, `module load ffmpeg`), but `gnu8` is a compiler of the old CentOS 7 system; on Puma check `module avail ffmpeg`. Without one, copy the frames home (IB/11's 51: 4.1 MB) and run `make_movie.py` there.
- The same page's encoding line, `ffmpeg -f image2 -i movie%04d.png -r 10 output.mp4`, puts the rate after the input: on IB/11's 51 frames it kept 22 (2.2 s) and wrote the 4:4:4 colour format many players refuse. `-framerate 10` before `-i` and `-pix_fmt yuv420p`: 51 frames, 5.1 s (chapter 30).

UArizona HPC documentation: the VisIt page, and "Operating System Updates" (GCC 8, `gnu8`, the old system's default compiler); `wsl/final/ch30_uahpc_ffmpeg.sh` (ffmpeg 6.1.1); the transfer host as in chapter 1.

A PARALLEL ENGINE (1/2)

```
visit -nowin -cli -np 4 -s visit_movie_frames.py -- viz_IB2d frames      # 4 engine processes
```

- VisIt 3.5.0's Ubuntu 24 tarball (the one measured) carries a parallel engine (`engine_par`) and its own MPICH (`mpirun`); `-np 4` starts `mpirun -np 4 engine_par` (four processes seen, plus `mpirun`).
- Measured on IB/11 (51 frames, 2D): 20.6 s serial, 20.0 s with 2 processes, 20.2 s with 4. A small 2D picture is drawn by the viewer alone (below VisIt's threshold of 2 million polygons for scalable rendering), so more engine processes do not speed it up. The frames were byte-identical with 2 processes and with 4 but one, which differed in 2 pixels, by 3 of 255 levels in green and blue.

`wsl/final/ch31_parallel.sh` (VisIt 3.5.0, one machine; `visit_movie_frames.py` with the `-engineargs` fix).

A PARALLEL ENGINE (2/2): WHERE IT HELPS, SCRIPTS, PUMA

- It should help where the engine reads and computes a lot (big 3D dumps, isosurfaces, queries over many cells): not measured here. `visit_queries.py` gave the same numbers with 2 processes (slide 19). `resample_export.py` with VisIt's distributed resample on (its setting until this chapter) exported each state in pieces with `-np 4`, two processes writing into one file; with it off (the script now), `-np 4` wrote the serial engine's file, value for value (exercise 5).
- VisIt passes its engine options to the script: `sys.argv` ends in `-engineargs -l;mpirun;-np;4 -launchengine localhost`. A script that reads `key=value` arguments must stop there (the `examples/analysis` scripts do now; before, they stopped with a `ValueError`).
- On Puma the engine would be launched inside the job's allocation (VisIt's launch options `-l srun`, `-p`, `-la`): not yet tried.

`wsl/final/ch31_parallel.sh` (VisIt 3.5.0, one machine; `visit_movie_frames.py` with the `-engineargs` fix); `resample_export.py` with and without the distributed resample: `wsl/final/ch31_resample_np.sh` (IBFE/10, one state, 48^3).

LONG RENDERS: MEMORY, CHUNKS, RESTARTS

- The 2026 runs on Puma (VisIt 3.4.2, large 3D IBFE output) saw the engine's memory grow with every time state, and processed those runs in chunks. Measured here in VisIt 3.5.0 on IBFE/10's flow tank (3D, 21 dumps, 95 MB), a slice of $|\boldsymbol{\omega}|$ saved at every dump, twice through the run: the engine held 118 MB at the first frame and 179–186 MB for the other 41, no growth; and 144–146 MB at every state while `resample_export.py` resampled and exported all 21 dumps, the kind of work that grew on Puma (`scratch/ch31/verify_agent/v4_ibfe_mem.sh`). It depends on the data, the plots and the version: watch it on yours.
- If it does grow (other plots, other readers, other versions), `visit -movie -enginerestartinterval n` restarts the engine every n images (chapter 30), and a script can take its dumps in pieces: `start=0 end=99`, then `start=100 end=199 first=100`, in two jobs or one after the other. `first=` numbers the second piece on from `f0100.png` (without it every piece starts again at `f0000.png`) and appends to `frames.csv`; `make_movie.py` takes the frames in name order.
- `sacct -j <jobid> --format=MaxRSS` after a job gives its peak memory: ask for a little more next time, not ten times more.
- A job that hits its time limit is cut off; the frames written so far stay. Give the next job `start=` and `first=` the first missing dump.

`wsl/final/ch31_memory.sh`; the 2026 Puma notes; the chunk keys of `visit_movie_frames.py`.

IBFE MESHES ON THE CLUSTER

```
# the 2020 tutorial's lines (3D: dX_0, dX_1, dX_2)
OpenDatabase(string_mesh)          # viz_IB3d/output.ex2
AddPlot("Subset", "Mesh", 1, 0)
SetActivePlots(1)
AddOperator("Displace", 0)
DefineVectorExpression("dX", "{dX_0,dX_1,dX_2}")
DisplaceAtts = DisplaceAttributes()
DisplaceAtts.factor = 1
DisplaceAtts.variable = "dX"
SetOperatorOptions(DisplaceAtts, 0, 0)
SubsetAtts = SubsetAttributes()
SubsetAtts.colorType = SubsetAtts.ColorBySingleColor
SubsetAtts.singleColor = (255, 255, 255, 255)
SubsetAtts.subsetNames = ("Whole mesh (Mesh)")
SubsetAtts.smoothingLevel = 2
SubsetAtts.pointSize = 0.05
SubsetAtts.pointType = SubsetAtts.Point
SetPlotOptions(SubsetAtts)
DrawPlots()
```

- An IBFE body's mesh is written once, in the reference configuration; the displacement `dX` moves it (chapter 28). "IBFE, unlike what we were doing in IBAMR, has meshes that will require a displacement operator to get them to move."
- `SubsetAtts.smoothingLevel = 2`: "This sets the smoothing level to High and gives a better-looking result than None or Fast (0 or 1 in the script), at least for the 3d coral meshes."
- In 2D VisIt 3.5 builds the vector `dX` from `dX_0`, `dX_1` itself; the definition above is the 3D case (chapter 28).
- The 2018 warning "The visit installed on killdevil cannot read .ex2 files" (VisIt 2.12.3) is history: VisIt 3.x reads ExodusII, so the export to VTK it required is not needed.
- `SetActivePlots(1)`: the plot's index in the list. "If you change the order in which you do things, you may need to change the number in the SetActivePlots() command." (2020)

G. Hobson (2020), slides 18–19, 31; "3B" (2018), slide 9; chapter 28. The 2020 lines run in VisIt 3.5.0 on IBFE/10's `output.ex2` (`scratch/ch31/verify_agent/v5_ibfe.sh`).

RESAMPLING AND VTK DATABASES ON THE CLUSTER

- The 2020 recipe: a Vector plot of `U`, the `Resample` operator with its box and samples (`useExtents = 0`, `startX ... samplesZ`, `is3D = 1`), `ExportDatabase` to VTK for all times; “Resampling and exporting to create these databases is much, much faster on the cluster.” Chapter 33 has it as `resample_export.py`, with VisIt 3.5’s export quirks.
- “It’s a good idea to check that the extents of your domain in each direction divided by the number of samples gives you the same number (i.e. you have a uniform grid).” And “match the number of vectors (nVectors) to the product of the number of xSamples, ySamples, and zSamples”.
- 2020: “on the cluster VisIt has trouble reading .VTK databases” (VisIt 3.1.2); the workaround was a `dumps.visit` in the VTK folder listing the file names. In VisIt 3.5.0 five exported files (`Omega_0000.vtk` ... `Omega_0004.vtk`) opened directly as the virtual database `Omega_*.vtk database`, five states, and also through a `.visit` list, with the same values. The list still works; here it was not needed (one machine; not tried on Puma).

G. Hobson (2020), slides 20–24; `wsl/final/ch31_vtkseries.sh` (VisIt 3.5.0, one machine).

AVERAGES OVER TIME ON THE CLUSTER

- The 2020 recipe: a Pseudocolor of `U_y` with a `Box` (`BoxAtts.amount = BoxAtts.Some`), a `QueryOverTime("Average Value", end_time=1000, ...)` (“You can set the end_time to a very high number and VisIt will automatically clamp to the number of timesteps you have”), and the curve saved as a `.curve` file.
- Chapter 29 measured this route in VisIt 3.5 on IB/11: the time query on a boxed plot came out 8 % below the same query frame by frame (the median ratio, 0.919; 0.69–0.75 on four frames of reversed flow); loop over the frames instead. `visit_queries.py` does both and writes `averages.csv`.
- “The .curve file will have a line at the very beginning with text preceded by a #. This will prevent it loading as a matrix in Matlab, but you can open the file and remove it manually.” `read_curve.m` and `read_curve.py` (chapter 29) read the file as it is, several curves included.
- In a batch job: `visit -nowin -cli -s visit_queries.py -- viz_IB2d/dumps.visit out box=...` (it ran with a parallel engine too, with the same numbers).

G. Hobson (2020), slides 26–28; chapter 29.

WAY 2: AN OPEN ONDEMAND DESKTOP

UArizona's VisIt page, step by step:

1. `ood.hpc.arizona.edu`, sign in, **Interactive Apps** → **Interactive Desktop**; hours, cores, your PI's group ("this workflow can also be performed with cpu only allocations"), **Launch**, then **Launch Interactive Desktop**.
2. On the desktop: right-click, **Open In Terminal**; start VisIt (the page installs a 3.2.1 tarball and runs `./visit -debug 5`; on Puma, `module load visit` and `visit`).
3. Open the data, plot, and **File** → **Save Movie**: **New simple movie**, format **PNG** ("A benefit to exporting individual frames as still images is it provides greater flexibility over quality options"), the right arrow, the frame range, the folder and base name, **Now, use currently allocated processors**.
4. Encode the frames with ffmpeg: use chapter 30's command, not the page's (slide 13).

The desktop is a compute node: everything of chapters 26–30 should work there as on your own computer (not tried for this chapter), with the data next to it. Its session ends with its time limit.

UArizona HPC documentation, VisIt page, "Using OOD" (its example: Ocelote with a GPU, VisIt 3.2.1). Not yet run on Puma for this chapter.

WAY 3: X11 FORWARDING

UArizona's visualization page, for VisIt ("GLX - Useful for VisIt"):

```
ssh -X <netid>@hpc.arizona.edu
shell -X
interactive -a your_group -x      # -x: graphics forwarding
module load cuda11                # provides GLX ("A GPU node is not required")
glxinfo                           # a test
glxspheres64                      # a good test of rendering back on the workstation
module load visit
visit
```

- On a Mac, an X server (XQuartz); its "Bad Value" error is fixed with `defaults write org.macosforge.xquartz.X11 enable_iglx -bool true`. On Linux the page gives an `AllowIndirectGLX` section for the X configuration.
- "in most cases the performance is not as good" as the OnDemand desktop: every picture travels over the network as drawing commands.
- No HPC VPN is needed for this: the recipe goes through the bastion host. The VPN is "primarily for applications that cannot navigate the bastion host for visualization, and because the performance is frequently better than tunneling through the Bastion host".

UArizona HPC documentation, Visualization page (read 17 October 2026). Not yet run on Puma for this chapter.

WAY 4: CLIENT–SERVER

The GUI runs on your computer; VisIt starts its metadata server and compute engine on the cluster. The dumps stay there; the plots' geometry travels and your computer draws it (the page: "there is no remote rendering of the visit window contents"; above 2 million polygons VisIt's scalable rendering, **Auto** by default, draws on the cluster and sends pictures). UArizona's VisIt page describes the set-up:

1. The HPC VPN (separate from the UA VPN; Duo), and SSH keys for `shell.hpc.arizona.edu`, "not the bastion hpc.arizona.edu host".
2. A VisIt server install on the cluster (the page: 3.1.4 in `/groups/<PIname>/visit_setup`) and a launcher script copied into its `3.1.4/bin` folder (`customlauncher`, from a gist the page links).
3. The same VisIt version on your computer, and the page's host profile `host_uahpc2.xml` in `~/.visit/hosts`.
4. **Options** → **Host profiles**: the absolute path of the server install, your NetID, and on the **Parallel** tab your PI's group as the **Bank**.
5. **Open** on the host **UAHPC2**, pick a launch profile; VisIt submits a Slurm job (`squeue` shows `visit.<netid>`) and connects when it starts.

UArizona HPC documentation, VisIt page, "Client/Server Configuration"; Visualization page (the HPC VPN "is separate from the UA VPN"); VisIt 3.5.0's `RenderingAttributes()` defaults.

CLIENT–SERVER: WHAT TO CHANGE FOR PUMA

- The page was written for VisIt 3.1.4 and the clusters of its time: its launch profile is `elgato`, its desktop example Ocelote; both clusters are retired. On Puma the launch profile needs Puma's partition (`standard`) and your group.
- The versions must agree at least in their first two numbers (NCSA's VisIt guide: "versions 2.12.0 and 2.12.1 are compatible but not 2.12.0 and 2.13.0"); the same release is safest. With Puma's module (3.4.2 in 2026) as the server, install 3.4.2 on your computer, or install your own server of the version you run locally.
- The host profile and the launcher script come from the page; read the launcher before you use it (the page offers a copy to inspect).
- This chapter did not test client–server: it needs the HPC VPN and an account. For most work way 1 (a batch script) or way 2 (the desktop) does the same with fewer parts; client–server pays off for interactive work on data too big to move, with a responsive local GUI.
- Support for this workflow: `vislab-consult@list.arizona.edu` (the page's address).

UArizona HPC documentation, VisIt page; Puma facts from the 2026 notes.

THE 2020 TUTORIAL'S OTHER LESSONS

- "Having a good structure for your input and output file directories will save you time and confusion": one folder per run, a `frames`, `curves` and `vtk` folder inside it, the script and its arguments in a README (chapter 29's data sheet).
- The loop of its slide 14 saves time states 1 to 30: it calls `TimeSliderNextState()` before each save ("if left as 0, the first timestep to be saved will be 1"). Save dump 0 too (chapter 30).
- Its view was set by numbers (`View2DAtts.windowCoords`, `viewportCoords`): the way to zoom in a script.
- "Restarting a job": a simulation, not VisIt; its restart line `mpirun ./main3d input3d $PWD/restart_IB3d 995000` is chapter 3's restart from a restart file. A restarted run writes `dumps.visit` afresh: rebuild the list from the `visit_dump.*` folders before a script reads the whole run (chapter 3; chapter 33).
- Resources it listed: the IBAMR tutorials, the VisIt user manual, and Shannon Jones's VisIt videos (credited in chapter 0.1, not used here).

G. Hobson (2020), slides 13–14, 30–32.

WHEN IT GOES WRONG ON THE CLUSTER

what you see	what to do
`module: command not found`, or no `visit` module	you are on a login node: `interactive` first (chapter 1)
the CLI stops at >>> after the script and waits	no `sys.exit()` at the script's end: add it (a batch job, with no input, quits there by itself)
"completed", no files	the output folder did not exist; a wrong path in the script (chapter 30)
`ValueError` on the arguments with `-np`	a script that does not stop at `-engineargs` (slide 14)
the picture is the whole domain, not the zoom	zooms made with the mouse "do not work on the cluster" (2020): set `windowCoords`
`OpenDatabase` fails on a recorded path	a Windows path from the recorder: use the cluster's (a `localhost:` in front of it is harmless)
the movie is short or will not play	the encoding line (slide 13; chapter 30)
the frames stop part way	the time limit: a new job with `start=` and `first=` the first missing dump
the job's exit status says it failed (or 250)	no guide: VisIt returned 250 from runs that ended with `sys.exit(0)`; count the frames
client–server never connects	VPN, SSH keys for `shell.hpc.arizona.edu`, the same version on both ends, the group in **Bank**
the dumps before a restart are missing from `dumps.visit`	the restarted run rewrote it: rebuild the list (chapter 3)

EXERCISES

1. On Puma, find the VisIt module (`module avail visit`) and an ffmpeg (`module avail ffmpeg`); note the versions in your data sheet.
2. Run `visit_movie_frames.py` on IB/01's output in an `interactive` session with `end=2`; copy the three frames home and check them.
3. Submit `visit_frames.slurm` for the whole run; read the `.out` file and `sacct`'s `Elapsed` and `MaxRSS`; encode the frames.
4. Run `visit_queries.py` in a batch job on IB/11 and compare `averages.csv` with chapter 29's numbers.
5. Time `resample_export.py` on a 3D run without `-np` and with `-np 4` inside an allocation of four cores, and compare the files as well as the times. On one machine, with VisIt's distributed resample on, `-np 4` wrote each state in pieces (a `.visit` list of three blocks under one name, and a `.vtk` of 40 % of the serial file's size that two processes had written into at once); the script now turns it off, and `-np 4` then wrote the serial engine's file. Does the parallel engine help there?
6. Make the same movie through an Open OnDemand desktop with the wizard (PNG frames) and compare it with the batch job's frames.
7. Write the README for your run folder: which script, which arguments, which VisIt version, which job made each file.

SUMMARY

- Four ways: a script in a batch job (the default), an Open OnDemand desktop (the GUI next to the data), X11 (quick looks), client–server (a local GUI on remote data; the most parts).
- On Puma: a `visit` module (3.4.2 in 2026) or your own Rocky 9 install; modules on compute nodes only; test interactively on a few dumps, then submit.
- A cluster script is a local script with the cluster's paths, a `sys.exit()`, existing output folders and a count of what it wrote.
- A parallel engine (`-np`) does not speed up drawing a small 2D frame; it is for heavy reading and computing. Scripts must stop reading arguments at `-engineargs`.
- VisIt 3.5 did not need the 2020 and 2018 workarounds here (VTK lists, the export of `.ex2`); UArizona's VisIt page needs Puma's names and a better ffmpeg line.

> Bring home frames, curves and resampled files, not dumps.

REFERENCES AND CREDITS

- G. Hobson, “Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster: A Tutorial” (Scripps Institution of Oceanography, UC San Diego, 23 September 2020); its UArizona version (“Based on a tutorial created by Gabrielle Hobson”; undated, its file from October 2023).
- L. A. Miller, “3B – Analyzing your IBFE simulation” (2018), “8 – Visit from command line and Python” (2018).
- UArizona HPC documentation, https://hpcdocs.hpc.arizona.edu: “Visualization”, “VisIt” and “CPUs and Memory” (read 17 October 2026); “Operating System Updates” for Puma’s Rocky Linux 9.
- VisIt 3.5.0, https://visit-dav.github.io/visit-website/ (the releases table; the GitHub release v3.5.0; the release notes of 3.2.0, for Python 3); H. Childs et al., “VisIt: An end-user tool for visualizing and analyzing very large data”, in *High Performance Visualization*, CRC Press (2012) 357–372.
- The runs: IB/11 (`TARGET_VP`), IBFE/10 (the flow tank), IBAMR 0.19.0, on a desktop PC under WSL; the scripts `wsl/final/ch31_*.sh` in the working folder.
- This chapter’s script: `examples/analysis/visit_frames.slurm`; it uses `visit_movie_frames.py` and `make_movie.py` (chapter 30).